МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего образования

«Ижевский государственный технический университет имени М.Т. Калашникова»

Кафедра «Программное обеспечение»

ОТЧЁТ
по лабораторной работе № 2
Архитектура и интерактивный прототип в коде
по дисциплине
«Проектирование и конструирование ПО»
Проект «Fit Nutrition Supply»

Выполнили студенты группы Б23-191-2: _____ подпись, дата Чипига А. А.

_____ подпись, дата Фазлыев А. А.

Проверил _____ подпись, дата Еланцев М. О.

Ижевск, 2026

СОДЕРЖАНИЕ

Введение..... 3
1 Уточнение концепции после замечаний..... 4
2 Структурная схема и технологии..... 5
3 Сценарии использования и интерфейс..... 7
4 Модель данных..... 10
5 Программные интерфейсы..... 12
6 Архитектурные решения..... 14
7 Проверка прототипа и покрытие требований..... 20
8 Работа с ИИ агентом..... 22
Заключение..... 24
Источники..... 25
Приложение А Схемы..... 26
Приложение Б Протокол проверки..... 35

ВВЕДЕНИЕ

Цель работы - спроектировать архитектуру Fit Nutrition Supply и создать исполняемый прототип с экранными формами и навигацией. Работу выполняют Фазлыев А. А. и Чипига А. А., группа Б23-191-2. Основание требований - сданный отчёт ЛР1, сохранённый в sources/submitted-lab1/. Номера требований в данном документе относятся именно к нему.

Прототип реализует связанный сценарий: параметры питания → рацион → рецепт и замена → список покупок → сравнение корзин. Это первый срез продукта, а не завершённая реализация всех требований ЛР1. Реальные интеграции магазинов и внешний ИИ не подключены. Учебные предложения пяти сетей и приближённые значения пищевой ценности явно обозначены в интерфейсе.

1 УТОЧНЕНИЕ КОНЦЕПЦИИ ПОСЛЕ ЗАМЕЧАНИЙ

Основная целевая аудитория - взрослые люди, заинтересованные в снижении веса и планирующие готовить самостоятельно. Приложение помогает связать калорийность меню с повседневными покупками. Возможность использования людьми с другими целями не меняет основную ЦА. Ижевск - город пилота, а не признак принадлежности к аудитории.

Бюджет и свободное время учитываются как ограничения: пользователь задаёт сумму на период и максимальное время приготовления блюда. Сервис не гарантирует, что любое сочетание ограничений выполнимо. При нехватке рецептов он сообщает об этом и сохраняет предыдущий план; при превышении бюджета показывает сумму превышения.

В дополнении к ЛР1 приведены таблицы аналогов с колонкой собственного продукта и уточнённое NFR-05 с фиксированными версиями браузеров. Сданные материалы сохранены без перезаписи, чтобы было видно исходное состояние и внесённые уточнения.

2 СТРУКТУРНАЯ СХЕМА И ТЕХНОЛОГИИ

Принят модульный монолит с браузерным интерфейсом и одним серверным процессом. Интерфейс отправляет JSON-запросы; сервер проверяет сессию и входные данные, вызывает прикладные функции и обращается к SQLite. Доменные расчёты вынесены из HTTP-обработчика в domain.py.

См. рисунок 1 - Структурная схема системы (приложение А).

Таблица 1

Подсистема	Ответственность	Реализация
UI	Формы, маршруты экранов, диалог рецепта, состояния ошибок	static/index.html, app.js, style.css
HTTP API	JSON, валидация запросов, принадлежность данных пользователю	server.py, Handler
Авторизация	Регистрация, bcrypt, cookie-сессии и ограничение попыток	server.py, таблицы users, sessions, login_attempts
Планирование	Норма калорий, подбор допустимых блюд, размеры порций	domain.py, validate_settings, generate_plan
Закупки	Список, запасы, упаковки, полнота, сравнение, бюджет	domain.py, shopping, calculate_baskets
Источник цен	Чтение предложений и обновление демонстрационного набора	prices.py, DemoPriceProvider
Хранение	Параметризованный SQL, транзакции, внешние ключи	server.py, connect, schema.sql

Подсистемы выделены по ответственности. Авторизация и SQL пока находятся в server.py; вычисления и источник цен вынесены отдельно. Это модульный прототип, а не набор независимых сетевых сервисов.

Стек: Python 3.12, http.server, sqlite3, SQLite, bcrypt 5.0.0, HTML5, CSS3, JavaScript ES2020, HTTP/JSON. Проверки: unittest и Playwright. Документация: Graphviz, draw.io, python-docx и LibreOffice; эти инструменты не нужны для запуска приложения.

В прототипе HTTP-сервер слушает только 127.0.0.1. Это локальная демонстрация. Публикация в интернете потребует сервера приложений и HTTPS; NFR-01 в части HTTPS пока не выполнено.

3 СЦЕНАРИИ ИСПОЛЬЗОВАНИЯ И ИНТЕРФЕЙС

3.1 Варианты использования

Актор - взрослый пользователь. Он регистрируется или входит, задаёт параметры, получает меню, открывает рецепты, выбирает замену, учитывает запасы и сравнивает закупку. Получение списка входит в состав сценария меню, а расчёт целых упаковок - в сравнение корзин. Источник учебных цен является внутренним компонентом, поэтому на диаграмме он не изображён как реальный внешний магазин.

См. рисунок 2 - Диаграмма вариантов использования (приложение А).

3.2 Основной сценарий

После регистрации пользователь проходит обязательный профиль: параметры, активность и цель, необязательные ограничения и сводку. Сохранение открывает дашборд. Отдельный экран плана принимает период 7-31 день, 3-5 приёмов пищи, бюджет периода, время и предпочтения. Параметры человека берутся из сохранённого профиля. Для каждого приёма выбирается допустимый рецепт, затем размер порции масштабируется под долю суточной энергии. Новый план записывается транзакцией только после успешного подбора всех позиций.

Из сохранённого плана формируется единый список, вычитаются домашние запасы, количества округляются до целых товарных упаковок. Полные корзины сортируются по сумме покупки. Неполная корзина показывается отдельно и не рекомендуется как самая дешёвая.

См. рисунок 3 - Основной сценарий (приложение А).

3.3 Альтернативный сценарий

Пользователь открывает блюдо и запрашивает альтернативы. Сервер учитывает тип приёма пищи, аллергены, исключённые продукты, время и повторы в выбранной календарной неделе. Подтверждённая замена меняет только одну позицию. Размер порции сохраняет энергетическую долю

заменяемого приёма; суточные значения, список и корзины пересчитываются. При отмене запрос изменения не отправляется.

В результате меню сгруппировано по дням; у блюда есть калории, оценка цены ингредиентов и кнопка замены. Отдельная корзина сравнивается с бюджетом, превышение выделяется красным. Кандидаты проверяются по калорийности одинаковой базовой порции: допустимо отклонение до 15%. Затем размер выбранной порции уточняется под энергетику приёма пищи.

Для каждого кандидата без записи в базу пересчитывается весь список и лучшая полная корзина. Разница цены показывается до выбора. «Выбрать» применяет замену, а «Сохранить план» фиксирует статус saved. При новой замене статус снова draft. Номер версии защищает от устаревшего окна выбора, конфликт возвращает 409. Полная логика: docs/lab2/replacement_logic.md; исходники: replacements.py и result.js.

Каталог может не дать три допустимые альтернативы, требуемые FR-07: в таком случае прототип честно показывает доступное количество или отсутствие вариантов. Условие «не менее трёх» требует расширения каталога и не объявляется выполненным для любого ввода.

См. рисунок 4 - Альтернативный сценарий замены блюда (приложение А).

3.4 Исключительный сценарий

Кнопка проверки сбоя моделирует отказ источника цен. Адаптер выбрасывает PriceSourceUnavailable, API возвращает 503 с кодом PRICE_SOURCE_UNAVAILABLE. Данные предложений не изменяются. Интерфейс оставляет прежние суммы и показывает понятное сообщение. Это воспроизводимый учебный сценарий, а не сообщение о реальном отказе сети магазина.

Ошибки ввода возвращаются с кодом 422 и словарём полей. Если при заданных исключениях не хватает рецептов, прежний план сохраняется. Приложение не снимает ограничения, не заполняет неизвестные цены нулями и не заменяет рецепт без подтверждения.

См. рисунок 5 - Сценарий ошибки источника цен (приложение А).

3.5 Предварительные экранные формы

До программирования агент подготовил схему пяти экранов. Фазлыев А. А. подтвердил их состав в чате 25.10.2026. Схемы и первый обзор сохранены коммитом 6249f18 до реализации.

См. рисунок 6 - Предварительные экранные формы (приложение А).

Настройки, меню, рецепт с заменой, список и корзины образуют основной сценарий. Вход и регистрация - вспомогательная форма перед ним. На малой ширине навигация и карточки перестраиваются.

3.6 Обновление потока от 26.10.2026

Пользователь явно задал обязательный онбординг из трёх шагов и сводки, затем дашборд и отдельный план. Профиль сохраняется в SQLite, а незавершённый ввод и номер шага восстанавливаются из браузерного черновика по аккаунту. После сохранения профиль становится доступен на дашборде. Прямой переход к генерации без профиля блокируется в UI и API.

Профиль и настройки плана разделены, чтобы новый период не перезаписывал параметры человека. Шкалы ограничивают диапазон и шаг, рядом показано текущее значение. Числовые поля как единственный способ ввода, одна длинная форма и пропускаемый профиль отклонены: они не соответствуют постановке этой итерации. Подробнее: docs/lab2/entry_flow.md и ADR-006.

См. рисунок 8 - Поток регистрации, профиля, дашборда и плана (приложение А).

См. рисунок 9 - Четыре экрана онбординга (приложение А).

4 МОДЕЛЬ ДАННЫХ

Схема хранения приведена в prototype/schema.sql. Для каждого соединения включаются внешние ключи. Идентификаторы предметных сущностей - UUID или стабильные идентификаторы каталога. Деньги хранятся целым числом копеек. Даты предложений и создания - строки ISO 8601 с часовым поясом; начало меню - календарная дата.

См. рисунок 7 - ER-модель прототипа (приложение А).

Таблица 2

Сущность	Первичный ключ	Связи и ограничения
users	id	email уникален; одна учётная запись имеет множество сессий и планов
sessions	token_hash	user_id → users; срок действия сессии
login_attempts	email, attempted_at	Журнал включает и неизвестные email, поэтому FK к users отсутствует
profiles	user_id	PK одновременно FK к users; не более одного текущего набора настроек
plans	id	user_id → users; снимок настроек и даты периода
plan_items	id	plan_id → plans, recipe_id → recipes; уникальна тройка plan_id, day, slot
recipes	id	Ингредиенты и шаги - JSON, состав хранится на базовую порцию
ingredients	id	Название, категория, единица, КБЖУ, метки аллергенов
stores	id	Справочник пяти сетей
offers	id	store_id → stores, ingredient_id → ingredients; фасовка, цена, наличие, дата, источник, город
pantry	user_id, ingredient_id	FK к users и ingredients; домашнее количество и отметка покупки

В рецептах ingredients_json содержит ссылки на идентификаторы ингредиентов. На ER-схеме эта связь показана пунктиром, поскольку SQL не

обеспечивает для неё внешний ключ. Каталог фиксирован и загружается из согласованного набора catalog.py. Перед вводом редактируемых рецептов потребуется нормализованная таблица recipe_ingredients и миграция данных. Это известный компромисс прототипа, а не заявленная реляционная целостность JSON.

У pantry отметка покупки пока относится к продукту пользователя, а не к отдельной закупке: она переносится на новый план. Это удобно для демонстрации сохранения, но перед рабочим выпуском нужна сущность shopping_list_item с отметкой в пределах конкретной закупки.

5 ПРОГРАММНЫЕ ИНТЕРФЕЙСЫ

5.1 Общие правила

API доступен по пути /api. Формат запросов и ответов - application/json, UTF-8. После входа браузер получает HttpOnly cookie session с SameSite=Strict. На сервере хранится SHA-256 токена, а не сам токен. Наличие авторизации проверяется до чтения пользовательского плана; чужая позиция не возвращается и не заменяется.

Коды ответов: 200 - успешное чтение или изменение; 201 - создан план; 401 - нет сессии или неверные данные входа; 403 - недопустимый Origin; 404 - ресурс не найден; 422 - ошибка полей или невыполнимые ограничения; 429 - превышен лимит попыток входа; 503 - отказ источника цен. Ответ об ошибке имеет поле error, а при ошибке ввода - fields. Тела запросов ограничены 64 КиБ. Пароли и тела запросов не записываются в журнал.

5.2 HTTP контракты

Таблица 3

Метод и маршрут	Вход	Результат
GET /api/session	Cookie, если есть	authenticated и email
POST /api/auth/register	email, password	Создание учётной записи, cookie; 422 при занятом email
POST /api/auth/login	email, password	Cookie; 401 при неверном пароле, 429 после пяти ошибок
POST /api/auth/demo	Пустой объект	Отдельная демонстрационная учётная запись без пароля
POST /api/auth/logout	Пустой объект	Удаление сессии и cookie
GET /api/state	Сессия	Профиль, каталог, текущий план, список и корзины
PUT /api/profile	Параметры человека, активность, цель, ограничения	Сохранённый профиль, profile_complete и состояние приложения
POST /api/plan	Период и настройки плана	201; параметры человека берутся из готового профиля
POST /api/plan/save	plan_id, expected_revision	Статус saved и дата; 409 при устаревшей версии

Метод и маршрут	Вход	Результат
GET /api/alternatives?item_id=…	ID позиции своего плана	Массив допустимых рецептов
POST /api/replace	item_id, recipe_id, expected_revision	Замена одной позиции, корзина и новая версия; 409 при конфликте
PUT /api/pantry	ingredient_id, amount, purchased	Сохранённый запас и пересчитанный снимок
POST /api/prices/refresh	simulate_failure: boolean	Новый снимок или 503; прежние цены при отказе сохранены

Пример запроса для готового профиля:

```
{"days":7,"meals":3,
"budget":3500,"minutes":30,"repeat":2,
"start_date":"2026-10-26",
"preferences":["no_fish"]}
```

Пример ошибки валидации:

```
{"error":"Проверьте настройки",
"fields":{"days":"Введите число от 7 до 31"}}
```

Пример строки расчёта предложения: ingredient_id = rice, needed = 600 г, pack_size = 500 г, price_kopecks = 10000. Результат: packs = 2, cost_kopecks = 20000, leftover = 400 г. Остаток считается относительно количества, которое нужно купить после вычета запасов.

5.3 Внутренние контракты

```
validate_settings(data: dict) -> dict
generate_plan(settings: dict, recipes: dict, ingredients: dict) -> list[dict]
nutrition(recipe: dict, ingredients: dict, scale: float = 1) -> dict
shopping(items: list, recipes: dict, ingredients: dict, pantry: dict) -> list[dict]
calculate_baskets(lines: list, stores: list, offers: list, budget: float) -> dict
DemoPriceProvider.read(db) -> list[dict]
DemoPriceProvider.refresh(db, simulate_failure: bool = False) -> None
```

Это контракты документации; функции Python не везде содержат аннотации типов в исходнике. validate_settings нормализует значения и возвращает target с калориями и БЖУ. generate_plan возвращает позиции day, slot, recipe_id, scale или выбрасывает DomainError. shopping возвращает required, at_home, needed, purchased и единицу. calculate_baskets возвращает baskets, recommended и saving_kopecks. Адаптер цен при отказе выбрасывает PriceSourceUnavailable до изменения базы.

6 АРХИТЕКТУРНЫЕ РЕШЕНИЯ

Приняты шесть ADR: архитектурный стиль, хранение, детерминированные расчёты, изоляция источника цен, безопасность учётных записей и разделение профиля с планом. Каждый документ содержит контекст с FR/NFR, альтернативы, решение, последствия и роль агента. Полные тексты включены далее; отдельные файлы находятся в docs/lab2/adr/.

ADR-001: Модульный монолит с браузерным интерфейсом

Статус

Принято

Контекст

FR-05–15 связывают меню, список и стоимость. Изменение блюда должно согласованно пересчитывать производные данные. FR-01 требует входа и сохранения данных пользователя, NFR-05 - браузерного интерфейса на нескольких ширинах. Команде нужен запускаемый срез без отдельной инфраструктуры для каждого компонента.

Рассмотренные альтернативы

1. Микросервисы: дают независимое развёртывание, но для первого среза добавляют сетевые отказы, отдельные конфигурации и сложность транзакций.

2. Полностью клиентское приложение: проще запуск, однако хранение только в браузере не обеспечивает данные учётной записи при входе из другой сессии на сервере.

3. Desktop-приложение: удобно локально, но требует упаковки под платформы и расходится с браузерным направлением исходной презентации.

Принятое решение

Один Python-процесс отдаёт статический HTML/CSS/JavaScript и HTTP API. Вычисления вынесены в domain.py, источник цен - в prices.py; транспорт и SQL пока находятся в server.py. В браузере нет второго независимого механизма расчёта корзины: он отображает согласованный снимок сервера.

Последствия

Упрощаются запуск, отладка и транзакционная запись плана. Усложняется дальнейшее выделение авторизации и хранения из HTTP-обработчика. Риск - рост server.py при добавлении функций; границы уже заданы контрактами. Стандартный HTTP-сервер годится для локальной демонстрации, но требует замены или production-обвязки при публикации.

Роль агента

Агент предложил серверное хранение после проверки FR-01 сданной ЛР1 и выбрал небольшой модульный монолит. Пользователь подтвердил набор экранов до программирования и уточнил аудиторию. Выбор конкретной серверной реализации выполнен агентом в рамках порученной разработки; отдельное сравнение технологий пользователем не приписывается.

ADR-002: Серверная SQLite и снимок настроек плана

Статус

Принято

Контекст

FR-01 требует сохранить данные учётной записи; FR-10 - запасы после повторного входа; FR-05 и FR-07 - целостное меню и замену одной позиции. Нельзя записать половину нового плана и потерять предыдущий при нехватке рецептов.

Рассмотренные альтернативы

1. PostgreSQL: подходит для многопользовательского развёртывания, но отдельный сервер и настройка избыточны для локальной защиты.

2. JSON-файл: прозрачен, но потребовал бы ручной координации одновременных записей, уникальности и связей.

3. localStorage: удобен для черновиков UI, но привязан к конкретному браузерному профилю и не выполняет серверную часть FR-01.

Принятое решение

Использовать SQLite через sqlite3. Включать foreign_keys на каждом соединении, передавать значения через SQL-параметры, закрывать соединения явно. Записывать план и позиции одной транзакцией; сначала полностью рассчитывать допустимое меню в памяти. Сохранять settings_json в плане, чтобы последующие изменения формы не меняли трактовку уже созданного периода. Состав фиксированного рецепта пока хранится в JSON.

Последствия

Упрощаются резервное копирование и запуск. Усложняются конкурентная запись при росте нагрузки и запросы по JSON-составу. Риск нарушения ссылок внутри JSON ограничен фиксированным каталогом, но не устранён SQL. Перед редактированием каталога нужны нормализация recipe_ingredients и миграции; дальнейшая база не обещана неизменной.

Роль агента

Агент сформировал ER по фактической схеме, отметил ограничение JSON и добавил API-проверку сохранения предыдущего плана. Тест на Windows обнаружил, что контекст SQLite сам по себе не закрывал соединение; агент ввёл явное закрытие. Пользователь не заявлял эти проверки как выполненные им лично.

ADR-003: Детерминированный расчёт меню и целых упаковок

Статус

Принято

Контекст

FR-02 задаёт пример нормы калорий; FR-05 требует ограничений меню; FR-07 - пересчёта после замены; FR-09–13 - списка и целых упаковок; FR-15 - проверки бюджета. В FR-17 уже указано, что числа для анализа должен считать код.

Рассмотренные альтернативы

1. Расчёт языковой моделью: гибкие объяснения, но недетерминированная арифметика и дополнительные расходы.

2. Полный математический оптимизатор: может учитывать много ограничений одновременно, но потребовал бы отдельной постановки задачи и усложнил первый срез.

3. Ручное меню без автоматического подбора: надёжно для демонстрации карточек, но не показывает основной сценарий FR-05.

Принятое решение

Использовать чистые Python-функции. Кандидаты фильтруются по составу, времени и повторяемости, затем порция масштабируется под энергетическую долю приёма. Список суммирует канонические количества, вычитает запасы. Для каждого предложения количество упаковок равно ceil(потребность / фасовка), стоимость считается целыми копейками. Рекомендуются только полные корзины; экономия сравнивается со следующей полной корзиной.

Последствия

Упрощаются воспроизводимость и проверка на примерах. Подбор не гарантирует оптимум бюджета или целевых БЖУ, а масштабирование может дать неудобные кулинарные порции. Эти ограничения видны в прототипе и требуют развития. Нельзя представлять приближённый учебный каталог как основу медицинских рекомендаций.

Роль агента

Агент написал расчёты и граничные тесты, включая неполную дешёвую корзину. Проверка примера ЛР1 выявила расхождение округления 234,5 г; агент согласовал правило half-up между сервером и интерфейсом. Пользователь уточнил фокус на похудении; это отражено в основном сообщении продукта, а не в вымышленных гарантиях результата.

ADR-004: Заменяемый адаптер демонстрационных цен

Статус

Принято

Контекст

FR-11 требует цены пяти сетей с датой и наличием; FR-12–13 используют фасовки; NFR-03 требует актуальности. Реальный доступ к данным сетей пока не подтверждён. Для ЛР2 нужен воспроизводимый сценарий интерфейса и обработки отказа источника.

Рассмотренные альтернативы

1. Сразу парсить пять сайтов: зависит от региона, защиты и формата страниц; доступ и условия сбора пока не проверены.

2. Убрать сравнение до появления API: исключило бы важный экран и ключевую ценность из демонстрации.

3. Захардкодить итоговые суммы в UI: быстро, но не проверяет упаковки, замену блюд и взаимосвязь меню со стоимостью.

Принятое решение

Создать DemoPriceProvider с методами read и refresh. Исходные предложения содержат сеть, ингредиент, размер упаковки, цену в копейках, наличие, дату, источник и город. Суммы вычисляются на этих данных. В интерфейсе постоянно указано, что цены тестовые. Моделируемый отказ выбрасывается до записи и возвращает 503; прошлые предложения остаются доступными. Будущий внешний адаптер должен сохранить эти контракты.

Последствия

Упрощаются повторяемая защита и тестирование ошибок. Нельзя делать вывод об экономии в настоящих магазинах или считать выполненным суточное обновление NFR-03. Риск - принять учебные данные за реальные; он снижен явной маркировкой и документированием. Доставка в первом сравнении не учитывается: сценарий обозначен как самовывоз.

Роль агента

Агент отделил интерфейс источника от расчётов и добавил тест неизменности корзин при 503. Решение опирается на ранее обсуждённый риск доступа к ценам. Пять реальных сетей не выдаются за интегрированные; агент не приписывает пользователю договорённости с магазинами.

ADR-005: Хеширование паролей и серверные сессии

Статус

Принято

Контекст

FR-01 предусматривает email и пароль. NFR-01 требует bcrypt cost 12, HTTPS и блокировку после пяти неверных попыток за 15 минут. Доступ к позиции плана должен проверять владельца, а демонстрация - не требовать публикации настоящих учётных данных.

Рассмотренные альтернативы

1. Пароль или обратимо зашифрованный пароль в базе: не соответствует NFR-01 и создаёт ненужный риск раскрытия.

2. JWT в localStorage: удобен для распределённой системы, но сложнее отзыв сессии, а токен доступен JavaScript.

3. Внешний OAuth-провайдер: убирает локальный пароль, но требует внешней настройки и расходится с конкретным требованием email/пароль первой версии.

Принятое решение

Хешировать пароль bcrypt с rounds=12. Создавать случайную cookie-сессию; сохранять только SHA-256 токена, срок действия и владельца. Cookie имеет HttpOnly, SameSite=Strict и Path=/. Проверять Origin изменяющих запросов и принадлежность данных. После пяти ошибок для email за последние 15 минут отклонять дальнейший вход до истечения окна. При выходе удалять серверную сессию. Демонстрационный вход создаёт отдельную учётную запись с отключённым паролем.

Последствия

Упрощается отзыв сессий и ограничивается доступ JavaScript к токену. Возникают расходы bcrypt и необходимость хранения сессий. Локальный HTTP на loopback не выполняет HTTPS-часть NFR-01; Secure-cookie, TLS и production-развёртывание должны быть добавлены перед доступом через сеть. Демонстрационные аккаунты требуют будущей очистки по сроку жизни.

Роль агента

Агент сверился с конкретным NFR-01 сданного отчёта и выбрал bcrypt вместо замены алгоритма без объяснения. Добавил проверку префикса хеша cost 12, входа второй сессии, изоляции другого пользователя и блокировки попыток. Выбор и его ограничения сформулированы агентом; отсутствующие production-проверки не приписаны студентам.

ADR-006: Обязательный профиль отдельно от плана

Статус

Принято

Контекст

FR-01 предусматривает регистрацию, FR-02 и FR-03 задают параметры человека и исключения, FR-05 и FR-15 относятся к плану на выбранный период. В задании пользователя от 26.10.2026 после регистрации обязателен профиль из трёх шагов и сводки, затем дашборд и отдельная настройка плана.

Рассмотренные альтернативы

1. Одна форма: меньше экранов, но смешивает устойчивые данные с периодом и не соответствует заданному потоку.

2. Пропускаемый профиль: быстрее открывает приложение, но допускает расчёт без обязательных исходных данных.

3. Только localStorage: не требует серверного маршрута, но готовый профиль оказывается привязан к одному браузеру и расходится с существующей авторизацией.

Принятое решение

Хранить профиль в profiles отдельно от снимка settings_json конкретного плана. Ввести PUT /api/profile и серверную проверку profile_complete перед генерацией. Черновик шагов хранить в localStorage по аккаунту; после успешного сохранения удалять его и переходить на дашборд. Для ограниченных чисел применять заданные пользователем шкалы, подтверждая значения серверной валидацией. В плане сохранять отпечаток значимых параметров профиля, чтобы не выдавать старый рацион за актуальный после изменения исключений.

Последствия

Повторный план не требует заново заполнять профиль. Возврат и перезагрузка не стирают черновик. Усложняется синхронизация черновика и готового профиля; теперь есть два состояния данных. Старый план после изменения параметров нужно пересоставить, хотя запись сохраняется. Дополнительный API требует проверки входных значений и принадлежности аккаунту. Автоматический расчёт для возраста 14-17 лет в этой итерации не добавляется.

Роль агента

Пользователь сформулировал явную задачу с четырьмя состояниями онбординга и дашбордом. Агент разделил модели, реализовал формы и серверную защиту, сохранил заданные диапазоны шкал, добавил тесты возврата, перезагрузки и обхода маршрута. Отпечаток плана предложен и реализован агентом как защита от скрытого использования старых ограничений. Числовые поля и одна общая форма рассмотрены при реализации и отклонены по требованиям этого запроса.

7 ПРОВЕРКА ПРОТОТИПА И ПОКРЫТИЕ ТРЕБОВАНИЙ

7.1 Функциональная проверка

Автоматические тесты проверяют не только наличие интерфейса, но и граничные условия расчётов: 31 день при пяти приёмах пищи, повторы в календарной неделе, запрет аллергенов, отсутствие допустимого меню, округление упаковок, неполные корзины и экономию относительно следующей полной корзины. API-тесты используют временную SQLite, проверяют хеш bcrypt с cost 12, вход другой сессии в ту же учётную запись, изоляцию чужих планов, блокировку попыток, сохранение плана после ошибки и отказ источника цен.

Браузерный сценарий проверяет демонстрационный вход, генерацию, замену, запасы, пять сетей, сохранение после перезагрузки и ошибку источника. Фактические результаты записываются в docs/lab2/test_results.md и машинный JSON. Наличие теста не подменяет проведённую проверку; результаты в документе приводятся после запуска.

7.2 Уточнение NFR-05

Целевая проверяемая матрица: Google Chrome 155.0.8059.85 на Windows; Mozilla Firefox 158.0 на Windows; Safari 26.0 на macOS. Основные экраны настроек, меню, списка и корзины должны быть доступны на ширинах 360, 768 и 1920 CSS px без горизонтальной прокрутки страницы. Номер ОС, версия браузера, ширина и результат заносятся в протокол. Это зафиксированные версии для приёмки, а не подвижное обещание поддержки «двух последних версий».

В текущем окружении фактически проверен Google Chrome 155.0.8059.85. Проверки Firefox 158.0 и Safari 26.0 остаются пунктами приёмки; они не объявлены пройденными. Мобильная ширина в настольном браузере проверяет компоновку, но не заменяет испытания на реальном телефоне.

7.3 Матрица покрытия

Таблица 4

Требования ЛР1	Состояние прототипа
FR-01	Регистрация и вход, серверное хранение и повторный вход реализованы. Доступ с другого устройства требует сетевого развёртывания
FR-02	Профиль: возраст 14-90, вес 40-200 с шагом 0,5, рост 140-220. Автоматический расчёт текущего прототипа доступен с 18 лет
FR-03	14 меток аллергенов, исключения и проверка состава реализованы для фиксированного каталога
FR-04	Несколько членов семьи пока не реализованы
FR-05, FR-06	7–31 день, 3–5 приёмов, масштабирование энергии, повторы; при нехватке блюд явная ошибка
FR-07	Подтверждаемая замена и пересчёт; число альтернатив зависит от каталога, гарантия трёх вариантов ещё не обеспечена
FR-08	Карточка, состав, шаги, КБЖУ и количество на приём; самостоятельный выбор числа порций ещё не реализован
FR-09, FR-10	Общий список, категории, запасы и купленные позиции реализованы; каталог использует канонические единицы
FR-11	Модель пяти сетей реализована на тестовых данных; реального получения цен нет
FR-12	Упаковки рассчитываются; сопоставление фиксировано в учебном каталоге. Ручной выбор товарного аналога ещё не реализован
FR-13, FR-15	Полнота, сумма, следующая полная корзина, дата и бюджет работают на тестовом наборе
FR-14, FR-16–19	Два магазина, экономные замены, внешний ИИ, шаблоны и учёт фактических расходов отложены
FR-20, FR-21	Заказ и оплата исключены из текущей версии по Won't
NFR-01	bcrypt и лимит попыток реализованы. HTTPS и production-развёртывание ещё не выполнены
NFR-02, NFR-04	Специальные нагрузочные испытания на условиях ЛР1 не проводились
NFR-03	Даты и сохранение цен при ошибке есть; реальное суточное обновление пяти сетей не реализовано
NFR-05	Зафиксированы версии. Пройденная часть матрицы указана в протоколе

Такое покрытие соответствует цели ЛР2 - исполняемый интерактивный прототип. Оно не означает выполнения всей спецификации будущего продукта.

8 РАБОТА С ИИ АГЕНТОМ

Следующая явная задача пользователя: «Реализуй экран результата после нажатия кнопки "Составить меню"». В постановке заданы дни, калории, цены, бюджет, сохранение и замены с влиянием на всю корзину. Агент реализовал экран, общий фильтр и предварительный пересчёт, затем проверил равенство прогноза и итоговой суммы. Activity-схема обновлена по фактическому процессу. Это отдельная реальная итерация разработки после онбординга.

Следующая итерация началась с явной постановки пользователя: «Спроектируй и реализуй поток входа в Fit Nutrition Supply». Пользователь задал регистрацию, три шага профиля со сводкой, дашборд и отдельный план, точные диапазоны шкал и правила переходов. Агент реализовал поток, сравнил способы хранения, сформировал две редактируемые схемы и добавил проверки. Постановка и результат этой итерации сохранены в docs/lab2/prompts/2026-10-26-entry-flow.md и docs/lab2/entry_flow.md.

Пользователь передал агенту сданные материалы и замечания. До начала кода агент запросил уточнение аудитории, согласование экранов и смысл замечания NFR-05. Пользователь заменил предложенную агентом группу студентов на людей с интересом к похудению, подтвердил экраны и уточнил, что нужны версии браузеров. Эти ответы изменили проект до реализации.

Дословные значимые фрагменты запроса: «надо уже вторую лабу делать - реализовывать начинать продукт»; уточнение ЦА: «ЦА - люди с интересом в похудании»; ответ по экранам: «Да, подходит»; ответ по NFR-05: «указать версии браузеров вроде». Пунктуация первой фразы приведена к читаемому виду без изменения смысла. Полная переписка не подменяется вымышленными промптами.

Агент самостоятельно сформулировал техническую подзадачу: реализовать согласованные экраны, серверную авторизацию и хранение; вынести вычисления в чистые функции; отделить демонстрационный

источник цен; сформировать схемы из описания компонентов и согласовать ER с SQL; проверить основной, альтернативный и исключительный сценарии. Это рабочая формулировка агента, она не приписывается студенту как дословный запрос.

В ходе проверки обнаружены и исправлены ошибки: число SQL-параметров при первичной загрузке каталога; расхождение банковского округления Python с примером углеводов из ЛР1; незакрытое соединение SQLite, мешавшее удалению временной базы в Windows; лишние контуры фокуса из-за CSS-селектора; доступное имя кнопок навигации с декоративным символом. После исправлений тесты запущены повторно.

Агент подготовил связанный первый срез, контракты, схемы и тесты. При этом каталог требует проверки, реальные сети ещё не подключены, а покрытие требований неполное. Неисполненные проверки обозначены в протоколе.

ЗАКЛЮЧЕНИЕ

Подготовлены структурная схема, use case, три диаграммы активности, ER-модель, контракты, экраны и шесть ADR. Прототип включает регистрацию, обязательный профиль, дашборд, отдельный план, замену блюд, список и пять учебных корзин. Документы и код находятся в одном репозитории. Следующий этап: проверенный каталог и реальный разрешённый источник цен.

ИСТОЧНИКИ

1. Задание ЛР2, предоставленное преподавателем: sources/lab2-assignment.txt.

2. Сданный отчёт ЛР1 и презентация: sources/submitted-lab1/.

3. Python sqlite3: https://docs.python.org/3/library/sqlite3.html.

4. bcrypt 5.0.0: https://pypi.org/project/bcrypt/5.0.0/.

5. Firefox 158.0, примечания к выпуску: https://www.firefox.com/en-US/firefox/158.0/releasenotes/.

6. Safari 26.0, примечания к выпуску: https://developer.apple.com/documentation/safari-release-notes/safari-26-release-notes.

7. Chrome, документация версий: https://developer.chrome.com/release-notes. Версия фактически установленного браузера зафиксирована автоматической проверкой.

Приложение А Схемы

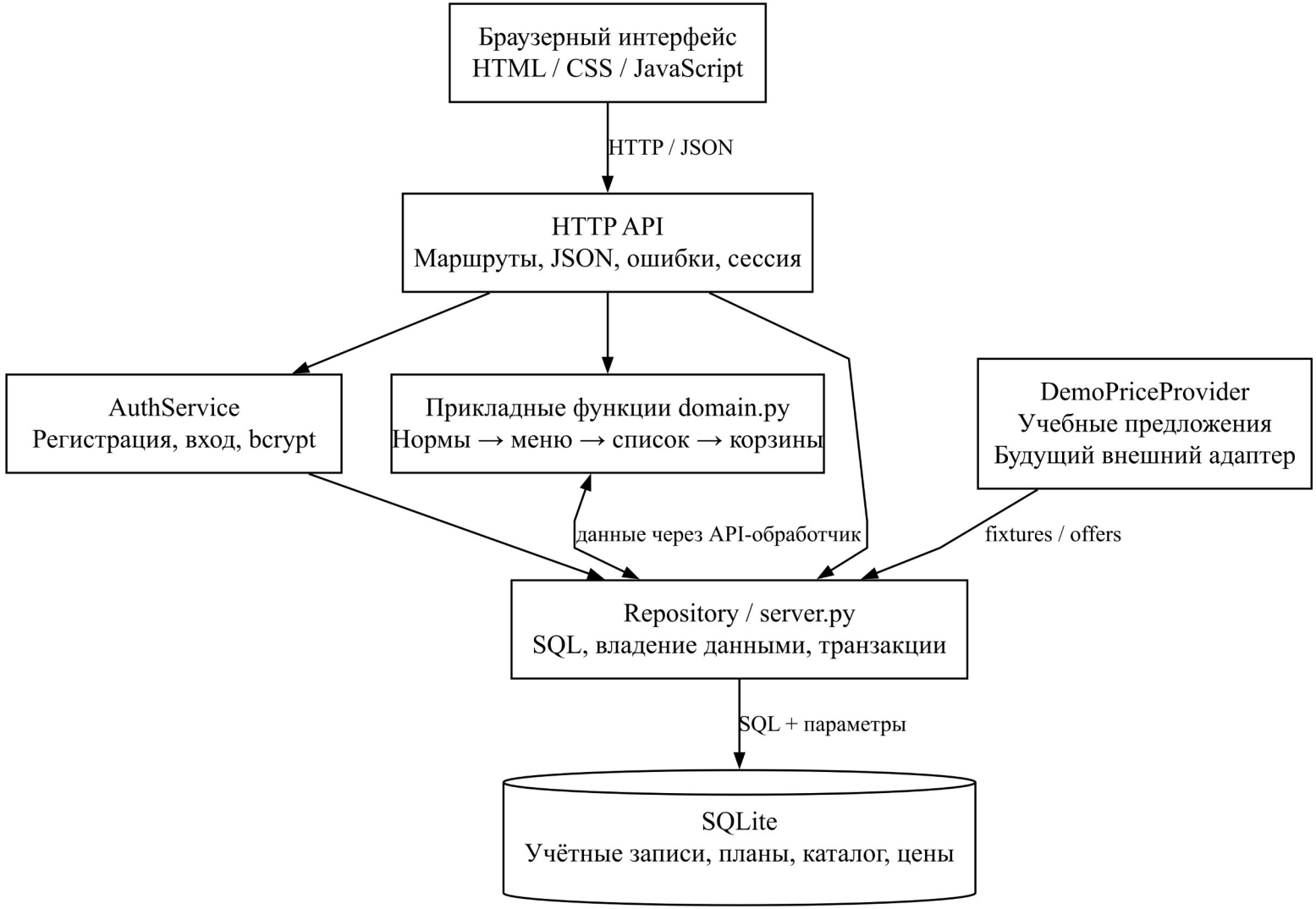


Рисунок 1 - Структурная схема системы

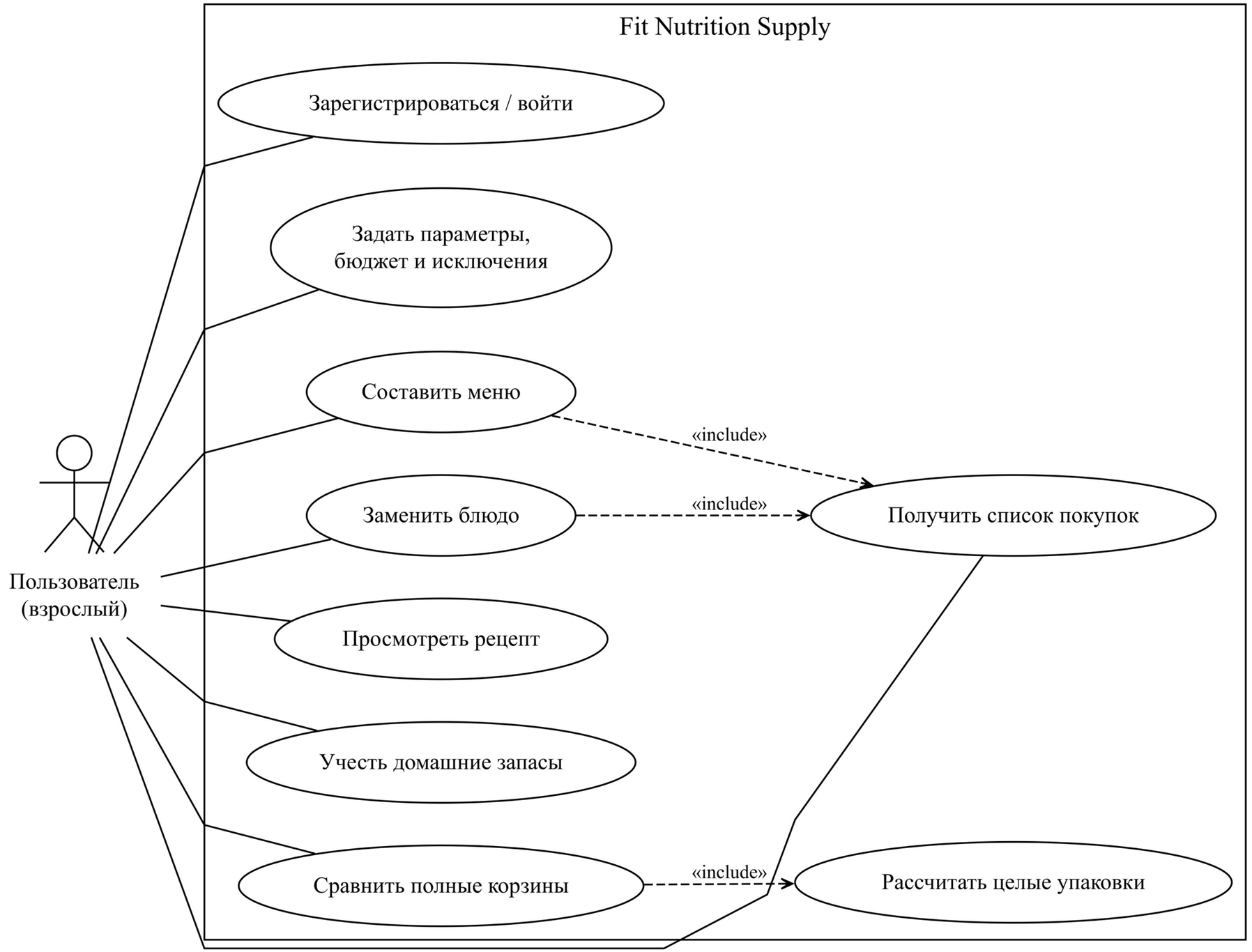


Рисунок 2 - Диаграмма вариантов использования

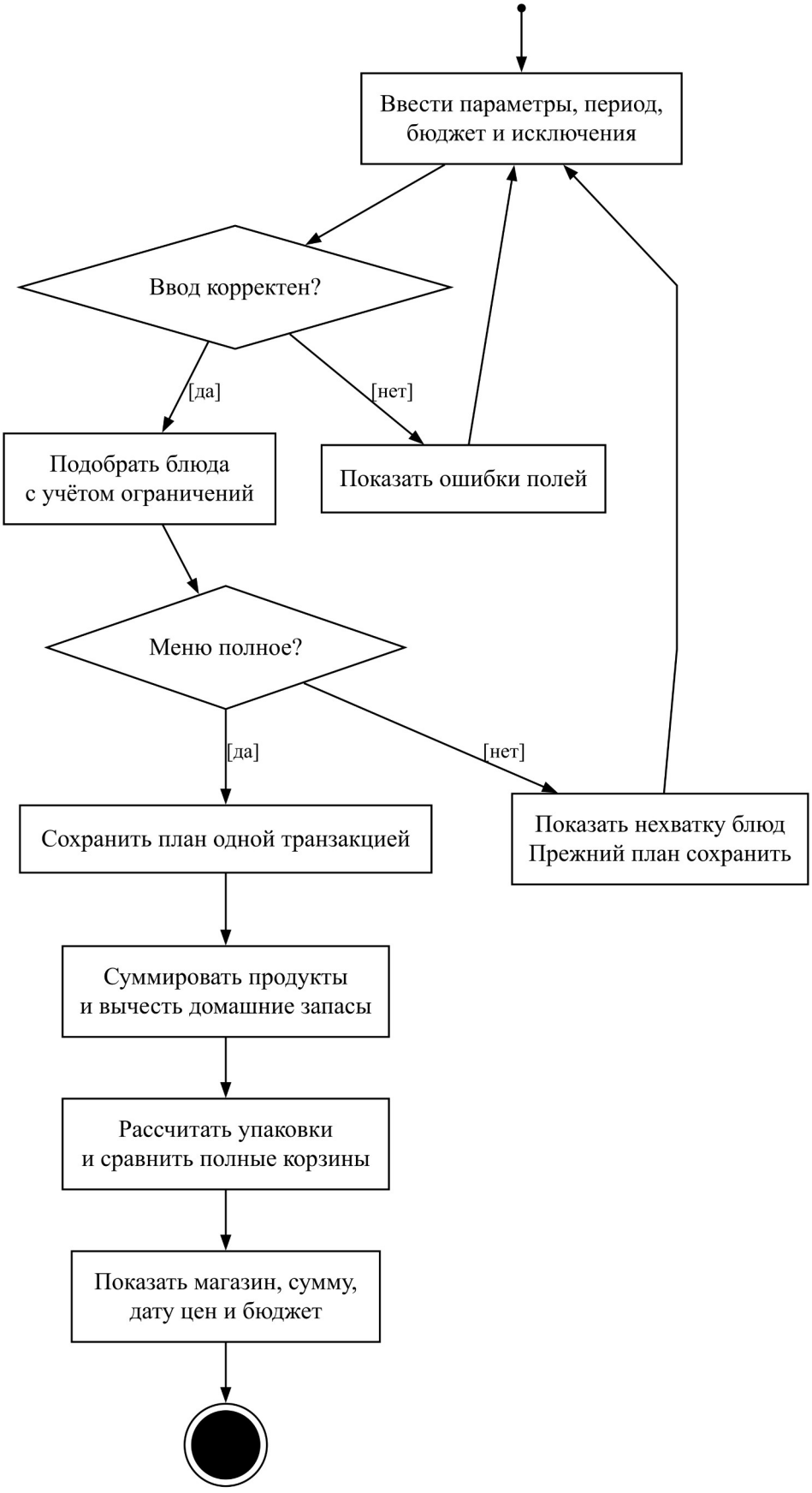


Рисунок 3 - Основной сценарий

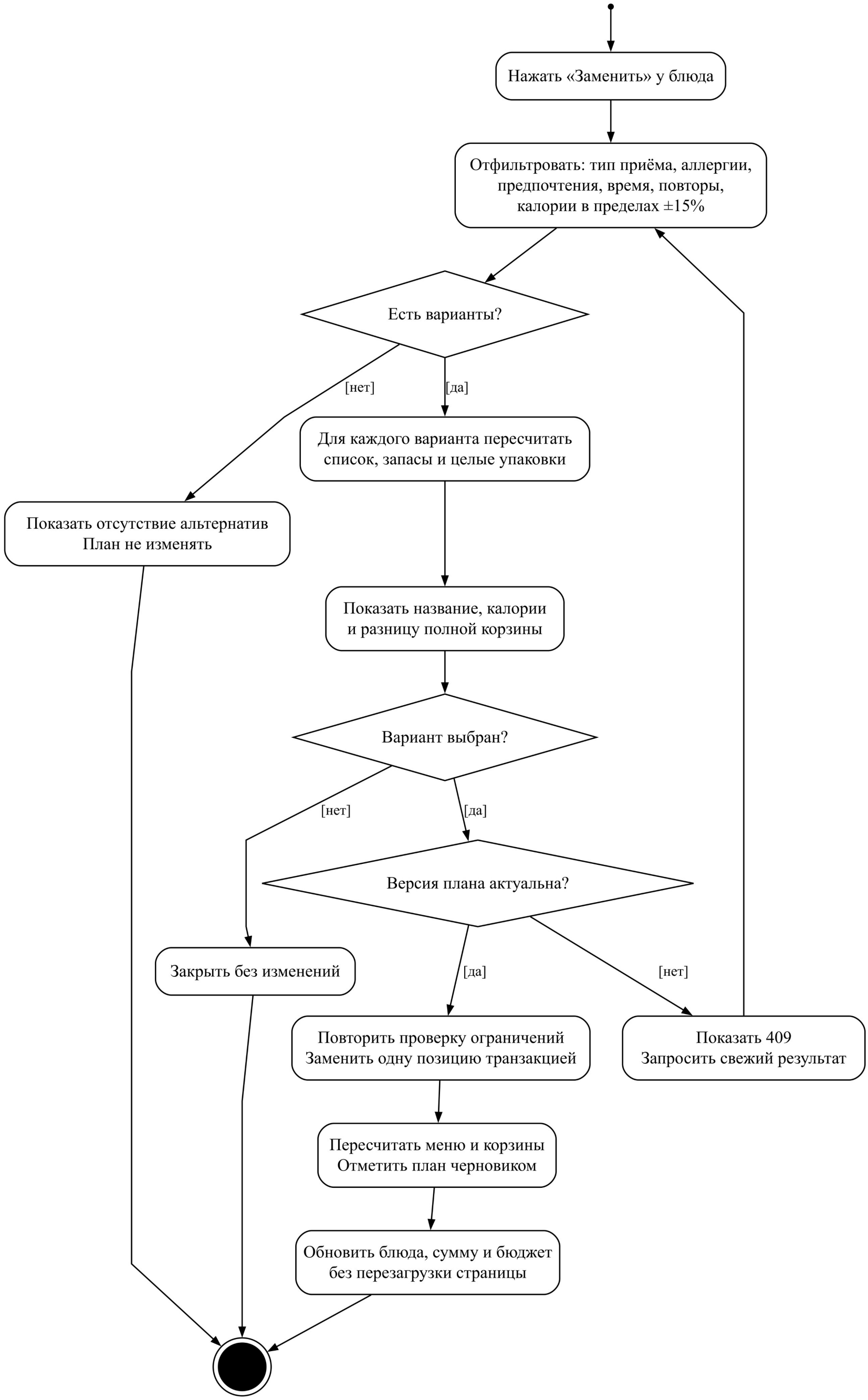


Рисунок 4 - Альтернативный сценарий замены блюда

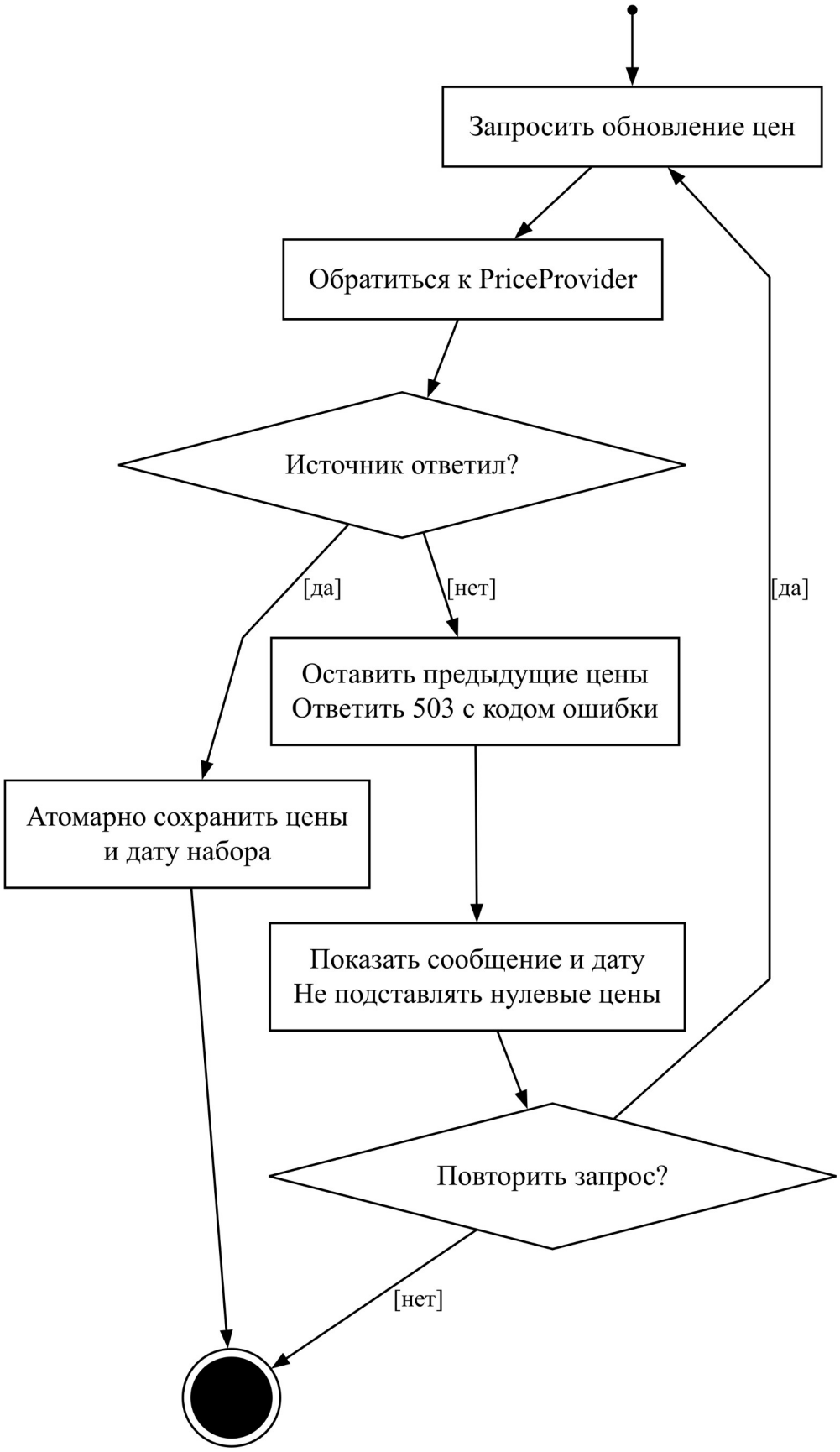


Рисунок 5 - Сценарий ошибки источника цен

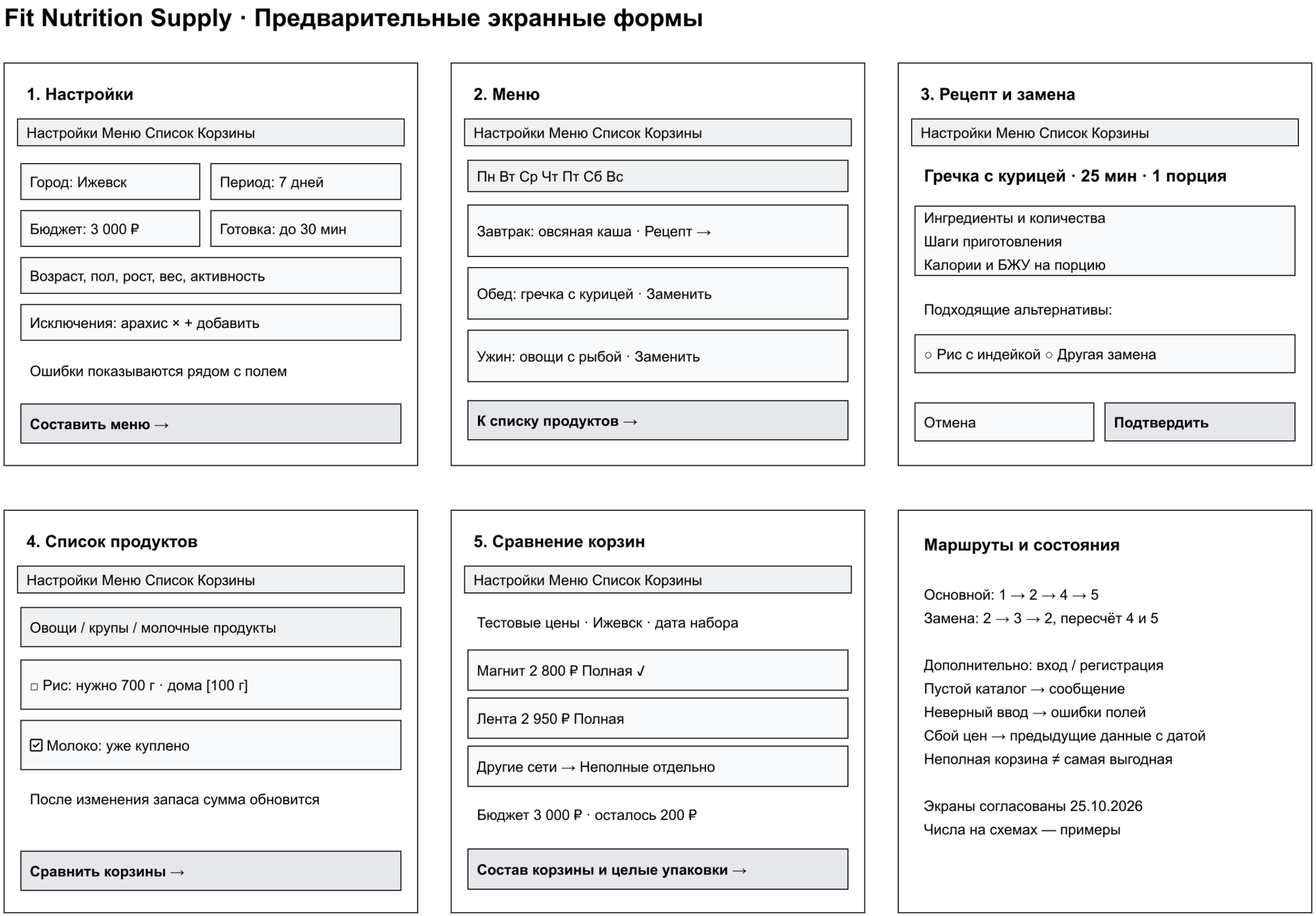


Рисунок 6 - Предварительные экранные формы

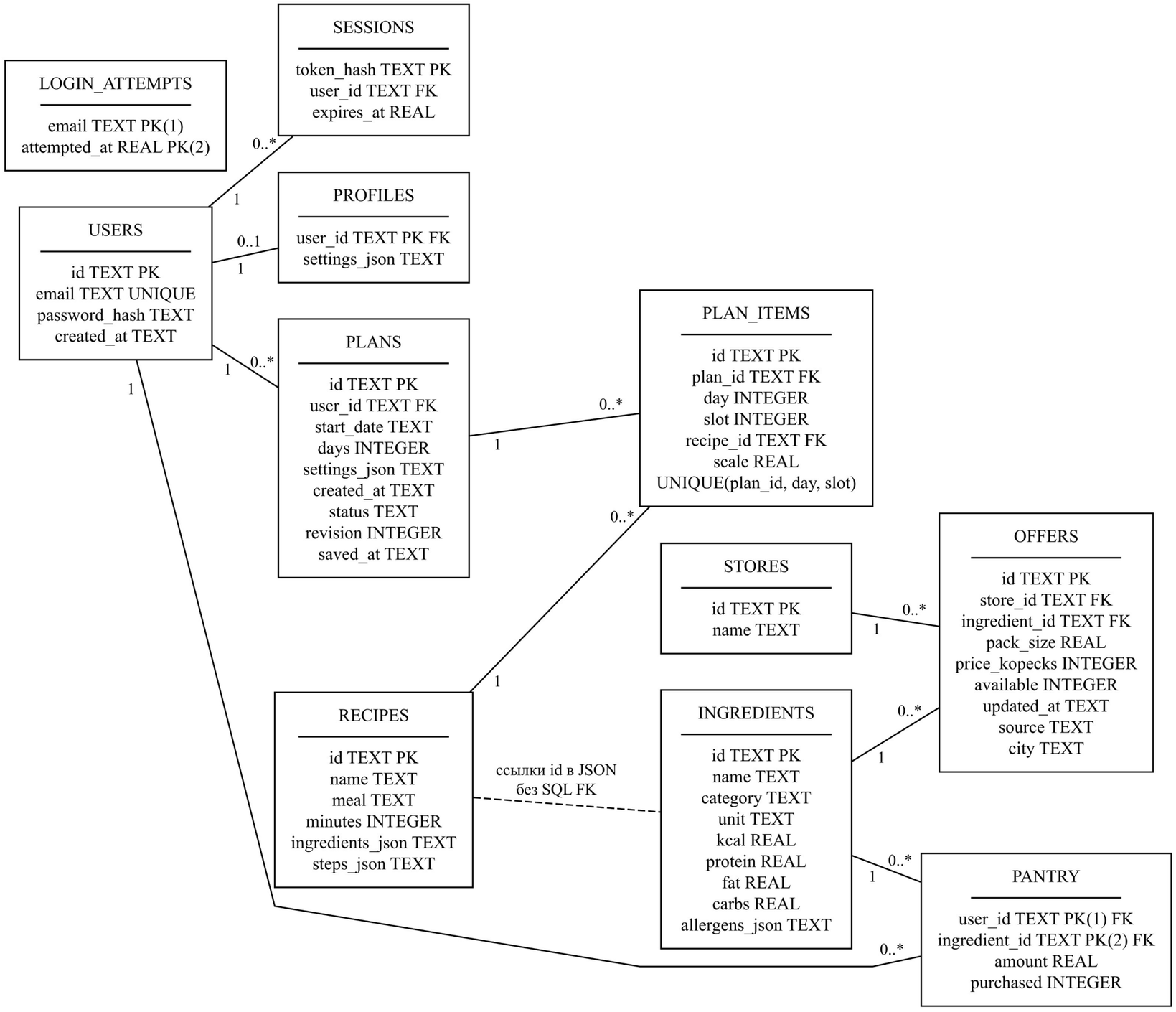
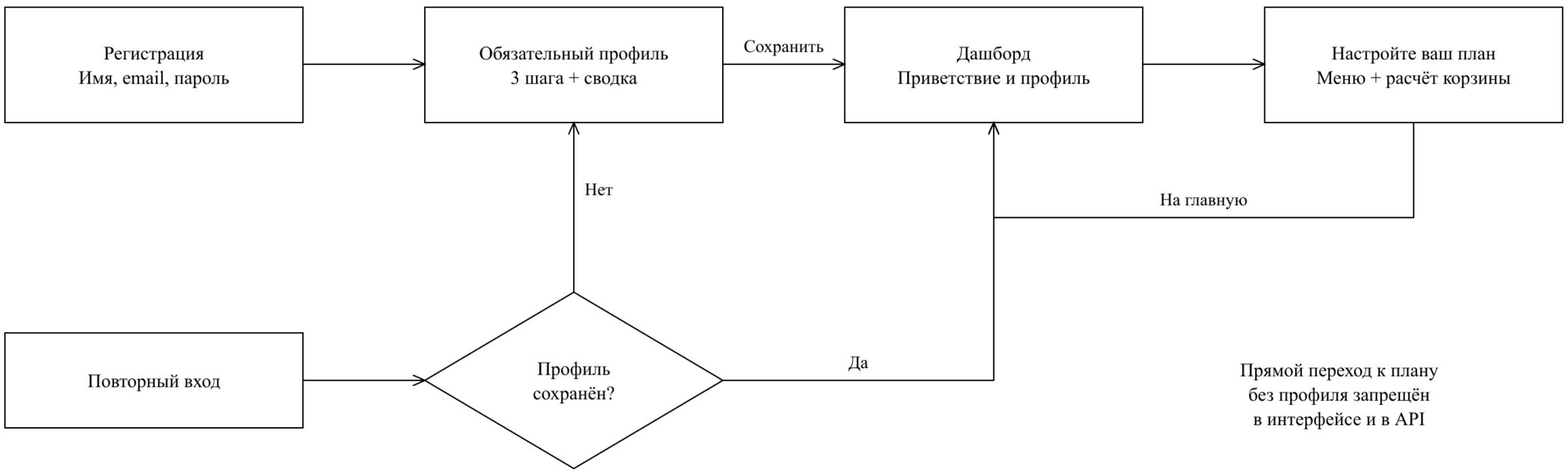


Рисунок 7 - ER-модель прототипа



Профиль хранится отдельно в SQLite. Черновик шагов хранится в браузере по аккаунту. Возврат и перезагрузка не стирают черновик. План сохраняет снимок параметров.

Рисунок 8 - Поток регистрации, профиля, дашборда и плана

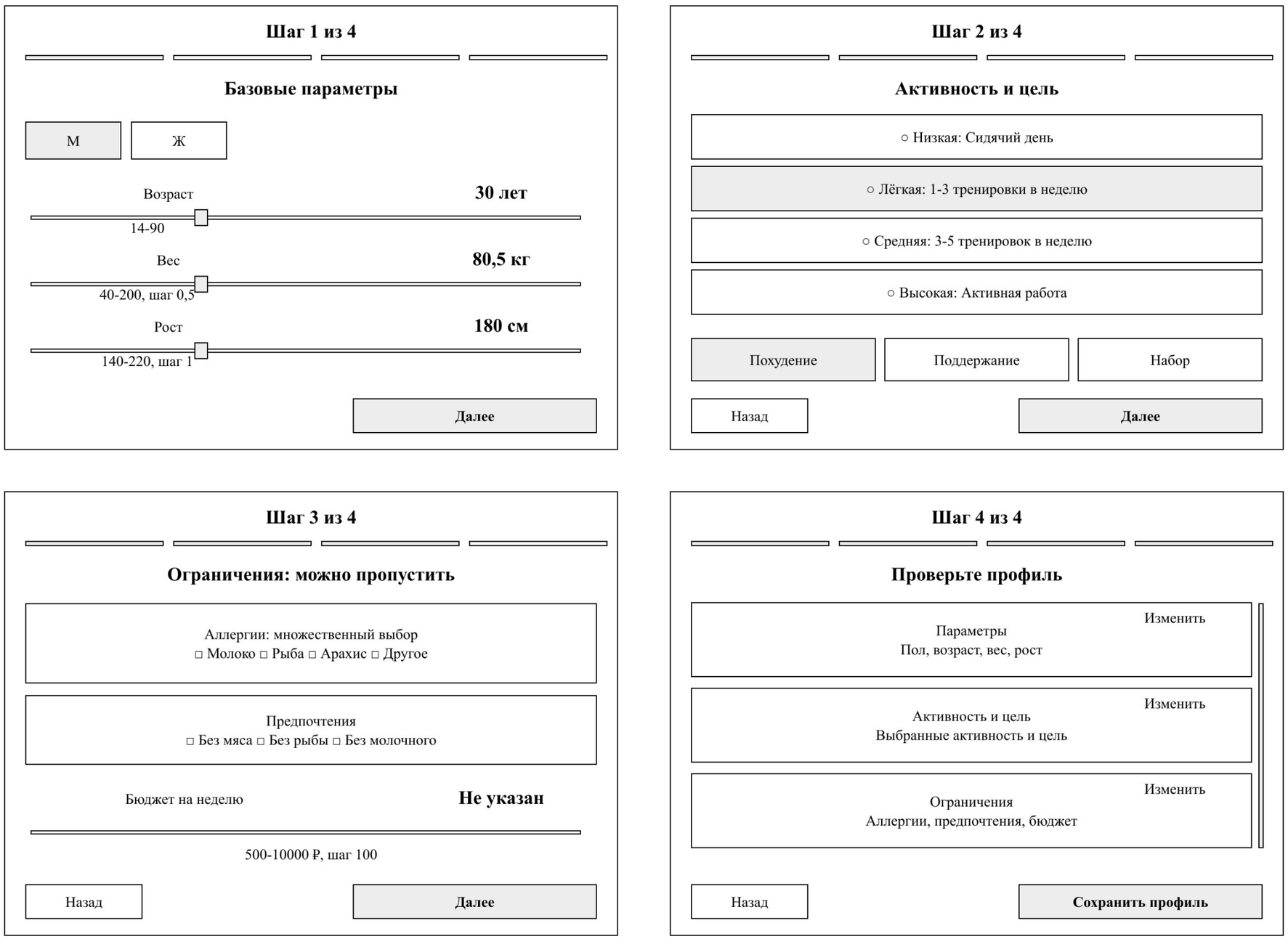


Рисунок 9 - Четыре экрана онбординга

ПРИЛОЖЕНИЕ Б ПРОТОКОЛ ПРОВЕРКИ

Дата: 26.10.2026. Среда: Windows 11 Pro, версия 10.0.26200; Python 3.12.14 и дополнительный прогон Python 3.10; Google Chrome 155.0.8059.85. Браузер управляется Playwright, приложение запущено на 127.0.0.1. Тестовые аккаунты и временные базы не содержат пользовательских данных.

Расчёты и HTTP API

Команда: python -m unittest discover -s prototype/tests -v из корня репозитория после установки prototype/requirements.txt.

Результат: 24 теста пройдены, ошибок и пропусков нет. Проверены 9 доменных и 15 API-сценариев. API-тесты поднимают отдельный сервер на свободном порту и используют временную SQLite.

Таблица 1

Группа	Проверка	Результат
Нормы	Пример ЛР1: 2345 ккал, 176/78/235 г БЖУ	Пройдено
Валидация	6/32 дня, дробный период, нулевой бюджет, NaN	Отклонены
План	7 дней × 3 приёма, энергия дня	21 позиция, отклонение менее 1 ккал на учебных данных
Длинный период	31 день × 5 приёмов, лимит повторов	155 позиций, не более 2 повторов в календарной неделе
Исключения	Арахис и орехи не попадают в план	Пройдено
Нехватка блюд	Ограничение 5 минут, невозможный основной приём	Явная ошибка без снятия ограничений
Упаковки	700 г нужно, 100 г дома, пачка 500 г за 100 ₽	2 пачки, 200 ₽, остаток 400 г
Полнота	Дешёвая неполная корзина рядом с полной	Неполная не рекомендуется
Экономия	Корзины 1250/1180/1210 ₽	Экономия 30 ₽ со следующей полной
Авторизация	Регистрация, повтор email, bcrypt cost 12, вход второй сессии	Пройдено
Ограничение попыток	5 неверных паролей и следующая попытка	Следующая попытка: 429
Изоляция	Запрос позиции чужого плана	404; чужой план не раскрыт
Выход	Повторное чтение state после logout	401
Отказ источника	503 при обновлении	Цены и суммы неизменны

Группа	Проверка	Результат
Ошибка плана	Неверный период и нехватка блюд	Предыдущий план сохранён
Origin	Изменяющий запрос с внешнего домена	403

Браузерный сценарий

Файл проверки: prototype/tests/browser.cjs. Машинный результат: evidence/browser-results.json. Пройдено: демонстрационный вход; заполнение формы; генерация; замена с подтверждением; учёт запаса; пять корзин; ошибка источника; перезагрузка с сохранением плана; отказ чтения состояния без сессии. Ошибок JavaScript в прогоне нет.

Таблица 2

Браузер	Настройки	Меню	Список	Корзины
Chrome 155.0.8059.85, 360 px	Пройдено	Пройдено	Пройдено	Пройдено
Chrome 155.0.8059.85, 768 px	Пройдено	Пройдено	Пройдено	Пройдено
Chrome 155.0.8059.85, 1920 px	Пройдено	Пройдено	Пройдено	Пройдено
Firefox 158.0, те же ширины	Не проверено	Не проверено	Не проверено	Не проверено
Safari 26.0, те же ширины	Не проверено	Не проверено	Не проверено	Не проверено

Горизонтальная прокрутка документа отсутствует на всех 12 проверенных сочетаниях. Компоновка проверяется в настольном браузере; реальная сенсорная клавиатура, устройства iOS/Android и Safari в этом окружении не проверялись.

Границы результатов

Result.cjs: прогноз цены, отмена, бюджет, сохранение и замена без перезагрузки проверены на 360/768/1920 px. Онбординг: 18 сочетаний без переполнения, возврат и восстановление черновика. Протоколы: evidence/result-results.json и evidence/onboarding-results.json.

NFR-02/04, реальные обновления NFR-03, Firefox и Safari не проверены. Каталог учебный, результаты не подтверждают рыночную экономию или медицинскую пригодность.